

An Intelligent Computational Task Scheduling Scheme for Satellite Edge Computing in a Satellite-Ground Cooperative Framework

IEEE Publication Technology Department

Abstract—With the continuous increase in the number of satellites, along with the expansion of application scenarios and the growth in data volume, current satellite data processing and computing methods have become inadequate. In response to these challenges, satellite-ground edge collaborative computing frameworks have been proposed in recent years. However, those frameworks suffer from limitations in intelligence, efficiency, and model refinement. Therefore, we propose an intelligent computational task scheduling scheme for satellite edge computing in a satellite-ground cooperative framework. This paper addresses the task scheduling problem in satellite scenarios by developing a cost model that considers thermal, latency, bandwidth, computation cost and task co-location¹ factors. A reward function is constructed based on this model. We proposed OSDQN-RSF (One Step Deep Q-Network with epsilon Reset, Selective update and Flushing replay buffer) algorithm, which is applied to optimize computing task scheduling between ground and onboard resources. The main contributions include: A cost modeling approach for satellite computing tasks, enabling quantitative evaluation of different scheduling strategies. Proposal of the OSDQN-RSF algorithm, which incorporates epsilon reset, selective network updates, and buffer reset mechanisms. Experiments show that OSDQN-RSF outperforms traditional methods in satellite task scheduling.

Index Terms—Satellite Edge Computing; Remote Sensing; Scheduling of Tasks Computing; Cost Modeling; Reinforcement Learning.

I. INTRODUCTION

IN recent years, the aerospace sector is witnessing remarkable growth and is experiencing an explosive expansion in its applications, ranging from global communications and Earth observation to navigation and remote sensing.

However, the current paradigm is characterized by a ground-centric, centralized processing architecture that relies heavily on manual task scheduling (Figure 1). In this model, satellites primarily function as data collection platforms, downlinking massive volumes of raw data to be processed exclusively in terrestrial data centers. This approach introduces critical limitations, including communication bottlenecks, significant latency, inefficient resource utilization, and an inherent lack of in-orbit intelligence and autonomy.

Manuscript created October, 2020; This work was developed by the IEEE Publication Technology Department. This work is distributed under the LATEX Project Public License (LPPL) (<http://www.latex-project.org/>) version 1.3. A copy of the LPPL, version 1.3, is included in the base LATEX documentation of all distributions of LATEX released 2003/12/01 or later. The opinions expressed here are entirely that of the author. No warranty is expressed or implied. User assumes all risk.

¹Task co-location refers to the degradation in task processing efficiency that occurs when multiple tasks are simultaneously processed by a single satellite. Given the precious nature of satellite computational resources, it is generally advisable to maintain the number of concurrently processed tasks on a satellite within a reasonable range.

Emiliano Casalicchio et al. formulated satellite scheduling as an optimization problem minimizing average task response time, demonstrating their decentralized algorithm surpasses centralized and distributed baselines in scalability and availability [1]. Junhong Kim et al. developed a MILP-based framework for multi-agile-satellite task scheduling, incorporating practical constraints and introducing a time-window pruning heuristic for near-optimal solutions [2], [3]. Shen Xin's group proposed a SAR satellite strip mosaic planning model optimizing orbit resources and imaging sequences under power constraints, solved via an improved PSO algorithm (LSDE-PSO) [4]; they also developed an SNN-PPO approach for multi-target imaging scheduling, leveraging temporal characteristics to reduce maneuvering costs and achieve global optimal sequencing [5]. Weisen Liu et al. presented Phoenix, an energy-efficient SEC scheduling framework that minimizes battery consumption while meeting task deadlines [6]. Qiangqiang Jiang et al. designed a scheduling model for HMPS that reduces makespan by 51% and energy consumption by 54%, while improving energy efficiency by 2.2× [7]; the team further introduced SECORS with its SEC-MPH algorithm, reducing computing time and energy consumption by 68.87% and 66.60% respectively while tripling energy efficiency [8].

Based on the reviewed literature, the previous optimization efforts in satellite-terrestrial computing systems are predominantly concentrated on the following key objectives:

1) *Multi-Satellite Collaboration*: Substantial work focuses on coordinated task planning and resource sharing within satellite constellations, including scheduling frameworks for agile satellites [3] and optimizing collaborative observation tasks [2].

2) *Temporal Performance*: A primary focus is the minimization of time-related metrics, including the average task response time [1], the overall makespan [7], and the onboard computing time [8].

3) *Resource and Energy Efficiency*: Given the critical constraints of spaceborne systems, significant attention is paid to optimizing energy usage. This is evidenced by efforts to minimize average battery consumption [6] and total energy consumption (TEC) [7], as well as to dramatically improve overall energy efficiency, often measured in specific units like pixels/W/s or pixels/W/ms [7], [8].

Prior work has centered on multi-satellite coordination, focusing on latency and thermal issues while overlooking bandwidth, task co-location, and computational costs. This has created a significant gap in the theory of intelligent single satellite-to-ground scheduling. Our proposed model addresses

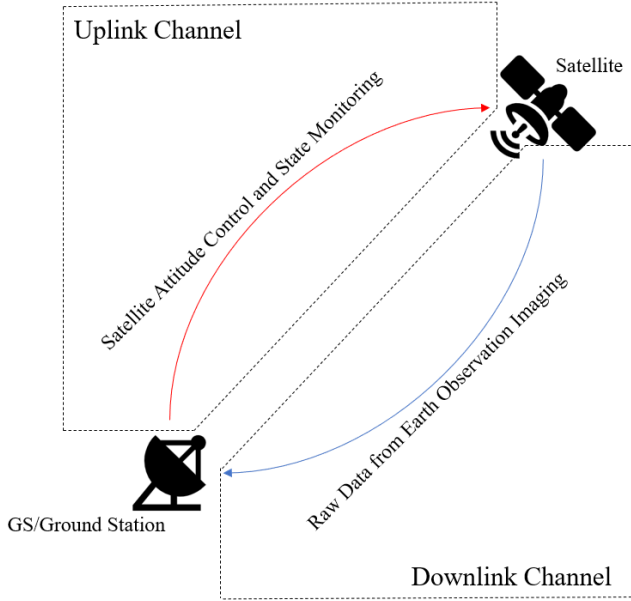


Fig. 1. The traditional manually scheduled satellite computing scheme.

this by providing a holistic cost-quantification framework tailored for this single satellite-to-ground context.

Our model incorporates critical factors such as heat generation, required floating-point operations (FLOPs), latency, co-location cost, and bandwidth usage. Each factor is modeled within the action space of satellite task scheduling, ultimately forming a comprehensive mathematical framework for quantitative cost modeling in satellite computational processing. The specific factors considered are summarized in Table I.

The selected factors—latency, thermal issues, bandwidth, co-location costs, and computational costs—constitute fundamental constraints in satellite-edge computing. Latency determines the viability of real-time applications [9]. Thermal issues are a primary hardware survival challenge in the space environment [10]. Bandwidth is a scarce physical resource that creates a bottleneck for data-intensive tasks [11]. Co-location costs arise from resource contention when hosting multiple services, degrading overall system utility [12]. Finally, computational costs are directly tied to the satellite’s finite energy budget, making computational efficiency synonymous with operational endurance [12]. The proposed model is fundamentally built upon the comprehensive consideration of these five critical factors.

The main contributions of our work are summarized as follows:

- 1) A cost evaluation framework for satellite computational tasks is established. The model incorporates key factors such as thermal impact, processing latency, energy consumption, and co-location overhead. This enables quantitative assessment of various task types and supports informed offloading decisions.
- 2) An enhanced algorithm named OSDQN-RSF is proposed, which introduces three key improvements: epsilon reset, selective updating of network parameters,

TABLE I
KEY COST FACTORS IN SATELLITE COMPUTATIONAL TASK MODELING

Cost Factor	Rationale
Thermal Output	Strict thermal management is essential in space environments to maintain satellite functionality and prevent system failures.
Computational Load	Limited onboard computing capacity necessitates offloading computationally intensive tasks to ground stations.
Processing Latency	Time-sensitive remote sensing applications require strict adherence to real-time processing constraints.
Co-location Overhead	Concurrent execution of multiple tasks on a single satellite leads to resource contention and reduced efficiency.
Bandwidth Usage	Precious uplink/downlink capacity between satellite and ground requires careful allocation and conservation.

and flushing mechanisms for the experience replay buffer. The method significantly improves training accuracy and demonstrates broad applicability in reinforcement learning. The reward function is constructed using the cost model, with task assignment strategies as action space and task parameters as the state space. The trained neural network can rapidly determine optimal scheduling policies.

II. NOMENCLATURE

TABLE II
NOTATION DEFINITION

Scheduling Strategy	State Symbol	State Description
Task processed on the satellite	Q_s	Heat cost
	$FLOPs_s$	Computational cost (floating-point operations)
	t_s^1	Task processing latency
	t_s^2	Result transmission latency
	$f(n)_s$	Co-location cost (when multiple tasks run simultaneously)
Task processed on the ground	B_s	Bandwidth occupation cost
	Q_g	Heat cost
	t_g	Data transmission latency
	B_g	Bandwidth occupation cost
Task pre-processed on the satellite and further processed on the ground	$Q_{s\&g}^{pre}$	Heat cost during preprocessing
	$Q_{s\&g}^{trans}$	Heat cost during transmission
	$t_{s\&g}^{pre}$	Preprocessing latency
	$t_{s\&g}^{trans}$	Data transmission latency
	$FLOPs_{s\&g}$	Computational cost (floating-point operations)
	$B_{s\&g}$	Bandwidth occupation cost

III. QUANTITATIVE COST MODELING

The primary objectives of quantitative cost modeling include: quantifying the impact of different factors on satellite computational costs and determine their respective weights in the final model based on their relative importance, assessing five factors across different tasks, and enabling dynamic task

- 2) Task data are pre-processed and segmented onboard the satellite (suitable for tasks such as object recognition or disaster detection, where large irrelevant portions of the image do not need to be transmitted). The segmented data are then transmitted to the ground for further processing.
- 3) All required data are directly transmitted from the satellite to the ground and processed.

This subsection develops cost models corresponding to these three different scheduling strategies, providing the mathematical foundation for applying reinforcement learning methods to task scheduling in the following section.

1) *Cost Modeling on the Satellite Side*: When a task is processed onboard the satellite, the total cost consists of the following components:

- heat generation cost during task processing,
- computational workload cost measured in floating-point operations,
- task processing time cost on the satellite,
- data transmission delay cost for returning the processed results to the ground,
- co-location cost when multiple tasks are processed simultaneously on the same satellite,
- bandwidth occupancy cost resulting from transmitting the task results to the ground.

Accordingly, the total cost in this scenario is modeled as:

$$\begin{aligned} \text{Cost}_s = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_s}{Q_{\text{MAX}}}\right) + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs}_s \\ & + \rho_{\text{time}} (\omega_{\text{proc}}t_s^1 + \omega_{\text{trans}}t_s^2) + \rho_{\text{coloc}}\omega_{\text{coloc}} f(n)_s \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_s}{B_{\text{MAX}}}\right). \end{aligned} \quad (6)$$

Here, Cost_s denotes the total cost of processing the task onboard the satellite; Q_{MAX} denotes the maximum allowable heat generation; B_{MAX} denote the maximum available downlink bandwidth; ρ_{heat} , ρ_{comp} , ρ_{time} , and others are dimensionless coefficients for normalization associated with each cost component; ω_{heat} , ω_{comp} , and others represent the weights associated with each cost component. For meanings of other cost variables please check Section II.

2) *Ground-Side Cost Modeling*: When tasks are processed on the ground, the primary costs mainly consist of the following components:

- the thermal cost associated with downlink data transmission from the satellite to the ground,
- the latency cost incurred during transmission,
- the bandwidth occupation cost.
- the computational consumption, heat generation, and latency during computational processes on ground station servers.

Accordingly, the cost in this scenario can be expressed as:

$$\begin{aligned} \text{Cost}_g = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_g}{Q_{\text{MAX}}}\right) \\ & + \rho_{\text{time}}\omega_{\text{trans}} t_g \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_g}{B_{\text{MAX}}}\right) \\ & + \text{cost}_{\text{ground_server}}. \end{aligned} \quad (7)$$

$\text{cost}_{\text{ground_server}}$ denotes the computational consumption, heat generation, and latency during computational processes on ground station servers.

3) *Satellite-Ground Collaborative Cost Modeling*: When a task is preprocessed onboard the satellite (e.g., image segmentation) and then transmitted to ground for further processing, the total cost consists of the following components:

- heat generation cost during onboard preprocessing,
- time cost for preprocessing onboard,
- computational workload cost (FLOPs) for onboard preprocessing,
- transmission delay for sending the preprocessed data to the ground,
- heat generation cost associated with data transmission from the satellite to the ground,
- bandwidth occupancy cost for downlink transmission.
- the computational consumption, heat generation, and latency during computational processes on ground station servers.

Accordingly, the total cost in this scenario can be expressed as:

$$\begin{aligned} \text{Cost}_{s\&g} = & \rho_{\text{heat}}\omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_{s\&g}^{\text{pre}} + Q_{s\&g}^{\text{trans}}}{Q_{\text{MAX}}}\right) \\ & + \rho_{\text{time}} (\omega_{\text{pre}}t_{s\&g}^{\text{pre}} + \omega_{\text{trans}}t_{s\&g}^{\text{trans}}) \\ & + \rho_{\text{comp}}\omega_{\text{comp}} \text{FLOPs}_{s\&g} \\ & + \rho_{\text{bw}}\omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_{s\&g}}{B_{\text{MAX}}}\right) \\ & + \text{cost}_{\text{ground_server}}. \end{aligned} \quad (8)$$

$\text{cost}_{\text{ground_server}}$ denotes the computational consumption, heat generation, and latency during computational processes on ground station servers.

This section develops cost models for three distinct task scheduling strategies: tasks processed entirely onboard the satellite, tasks processed on the ground, and tasks preprocessed onboard and then transmitted to the ground for further processing. These models provide the mathematical foundation for applying reinforcement learning methods to satellite task scheduling in subsequent sections.

IV. OSDQN-RSF ALGORITHM DESIGN

The core objective of the satellite computational task scheduling problem is to minimize total cost—encompassing factors like thermal and latency—by selecting an optimal scheduling policy. This problem is inherently a decision-making process: for each state, defined by a multi-dimensional tuple (e.g., current thermal load, latency), an action must be

chosen from a high-dimensional action space to minimize the long-term cost. Given the characteristics of satellite computational task scheduling, this study adopts reinforcement learning as the solution methodology. Reinforcement learning is particularly suitable for decision-making tasks in dynamic environments, where feedback is obtained through a reward mechanism. Therefore, it provides an appropriate and effective framework for the scheduling problem addressed in this research.

This section builds upon the quantitative cost modeling of satellite computational tasks presented in Section III. We proposed an algorithm (One Step Deep Q-Network with epsilon Reset, Selective update and Flushing replay buffer, OSDQN-RSF) to address the satellite computational task scheduling problem. This section also focuses on modeling the state space, action space, and reward function of the algorithm employed in this study. Furthermore, the section introduces the network architecture and related design considerations used for implementing the proposed approach.

A. Modeling of State Space, Action Space, and Reward Function

In Section III, the various costs associated with satellite computational tasks were mathematically modeled, and the functional relationships between each cost component and the total cost were explicitly defined. Building upon this quantitative cost modeling, this section defines the state space, action space, and reward function of the algorithm employed in this study.

1) *State Space*: The state space serves as the basis for decision-making of reinforcement learning, which in this study corresponds to the various cost factors (e.g., heat, latency, etc.) defined in Section III. Since the three scheduling strategies for satellite computational tasks involve distinct heat, latency, and other cost components, these must be represented as different states. Based on the cost factors outlined in Section III, the states defined in the state space are summarized in Table II.

In total, the state space comprises fifteen distinct states with continuous values. Accordingly, the state space can be characterized as fifteen-dimensional and continuous.

2) *Action Space*: The action space of the algorithm corresponds to the three satellite computing task scheduling strategies described in Section III: (1) processing the task onboard, (2) processing the task on the ground, and (3) pre-processing the task onboard and subsequently transmitting it to the ground for further computation. These three strategies constitute the three actions within the action space.

3) *Reward Function*: The reward function corresponding to each action of the algorithm is defined as the negative of the cost derived from the cost quantification model introduced in Section III. This formulation is adopted because the objective is to select the action that minimizes the overall cost. By taking the negative cost as the reward, the algorithm ensures that higher costs lead to lower rewards, thereby aligning the decision-making process with the goal of cost minimization.

Since the ground has practically unlimited computational resources compared to the satellite, and the computational cost

on the ground is not the main focus of this work, the ground-side computation cost is not considered here. That is, we do not take $\text{cost}_{\text{ground_server}}$ in Equation 7 and Equation 8 into consideration when building reward function.

Specifically, the reward function for processing a task using the onboard lightweight remote sensing model is defined as follows:

$$\begin{aligned} \text{reward}_s = -\text{Cost}_s = - & \left(\rho_{\text{heat}} \omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_s}{Q_{\text{MAX}}}\right) \right. \\ & + \rho_{\text{comp}} \omega_{\text{comp}} \text{FLOPs}_s \\ & + \rho_{\text{time}} (\omega_{\text{proc}} t_s^1 + \omega_{\text{trans}} t_s^2) \\ & + \rho_{\text{coloc}} \omega_{\text{coloc}} f(n)_s \\ & \left. + \rho_{\text{bw}} \omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_s}{B_{\text{MAX}}}\right) \right) \end{aligned} \quad (9)$$

The reward function for processing a task using the ground-based large-scale remote sensing model is defined as:

$$\begin{aligned} \text{reward}_g = -\text{Cost}_g = - & \left(\rho_{\text{heat}} \omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_g}{Q_{\text{MAX}}}\right) \right. \\ & + \rho_{\text{time}} \omega_{\text{trans}} t_g \\ & \left. + \rho_{\text{bw}} \omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_g}{B_{\text{MAX}}}\right) \right) \end{aligned} \quad (10)$$

Finally, the reward function for pre-processing a task with the onboard lightweight model and subsequently transmitting it to the ground-based large-scale model for computation is defined as:

$$\begin{aligned} \text{reward}_{s\&g} = -\text{Cost}_{s\&g} = - & \left(\rho_{\text{heat}} \omega_{\text{heat}} \tan\left(\frac{\pi}{2} \cdot \frac{Q_{s\&g}^{\text{pre}} + Q_{s\&g}^{\text{trans}}}{Q_{\text{MAX}}}\right) \right. \\ & + \rho_{\text{time}} (\omega_{\text{pre}} t_{s\&g}^{\text{pre}} + \omega_{\text{trans}} t_{s\&g}^{\text{trans}}) \\ & + \rho_{\text{comp}} \omega_{\text{comp}} \text{FLOPs}_{s\&g} \\ & \left. + \rho_{\text{bw}} \omega_{\text{bw}} \tan\left(\frac{\pi}{2} \cdot \frac{B_{s\&g}}{B_{\text{MAX}}}\right) \right) \end{aligned} \quad (11)$$

The meanings of the variables appearing in these three reward functions are consistent with the explanations provided in Section II and Section III. At this point, the reward functions corresponding to the three actions in the action space are fully defined.

B. Network Architecture and Design Considerations

Considering that the state space in this study consists of fifteen continuous-valued variables, a relatively deep network with a larger number of parameters was designed, as illustrated in Figure 3, to adequately capture the complex mappings during training.

In addition to fully connected layers, a Dropout layer [13] is introduced. During training, Dropout randomly masks a subset

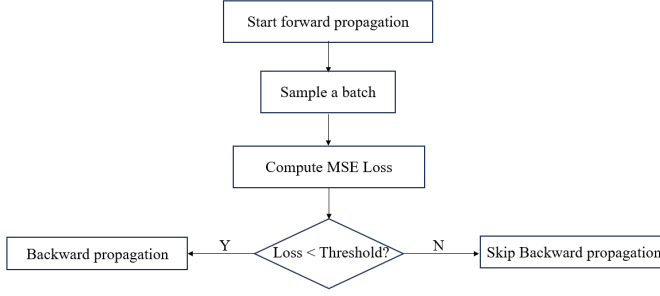


Fig. 6. The selective update mechanism in the OSDQN-RSF algorithm.

of a randomly sampled mini-batch may increase significantly. Learning from such batches is undesirable, as it implies that many state-action pairs in the batch deviate substantially from the mappings the network has already learned, with exploratory data dominating the update and impairing accuracy.

To address this issue, after epsilon reset we impose a loss threshold on each mini-batch (as shown in Fig 6): only those batches with a loss below the threshold are used for backpropagation. This strategy effectively reduces the negative impact of epsilon reset on network accuracy, ensuring that the model retains good generalization ability while maintaining high accuracy.

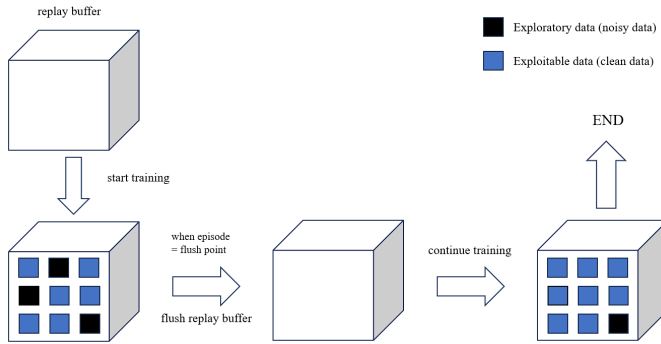


Fig. 7. Flushing replay buffer mechanism in the OSDQN-RSF algorithm.

4) *Flushing Replay Buffer*: The replay buffer is a key component of the baseline DQN algorithm, designed to store the agent's experiences during interaction with the environment. Each entry typically contains the current state, the action taken, the corresponding reward, and the next state. By sampling mini-batches of past experiences, DQN reduces data correlation, improves the stability and efficiency of learning, and helps prevent overfitting. In essence, the replay buffer enables the agent to revisit and leverage past experiences more effectively, thereby optimizing decision-making in complex environments.

In the proposed OSDQN-RSF algorithm, after epsilon reset has been applied and subsequently decayed to a minimal value (typically 0.01), the network is expected to have already acquired sufficient generalization ability. At this stage, we introduce the flushing replay buffer mechanism, where the replay buffer is cleared (as shown in Fig 7). This is motivated

by the fact that the buffer still contains a large proportion of random exploratory actions, which negatively impact accuracy. Following the flush, the replay buffer is refilled using the final minimal epsilon value: with probability ϵ the agent explores randomly, and with probability $1 - \epsilon$ it exploits the trained policy to select actions. This process greatly reduces the amount of noisy or uninformative data in the buffer and significantly improves accuracy once the network has attained adequate generalization capability. Compared with the baseline DQN, this mechanism provides a substantial performance enhancement.

Algorithm 1: OSDQN-RSF

```

1 Initialize replay memory  $\mathcal{D}$  to capacity  $N$ ;
2 Initialize action-value network  $Q$ ;
3 Initialize target network  $Q_{\text{target}}$  with parameters of  $Q$ ;
4 for  $episode = 1$  to  $M$  do
5   Initialize sequence  $s_1 = \{x_1\}$  and preprocessed
     sequence  $\phi_1 = \phi(s_1)$ ;
6   for  $t = 1$  to  $T$  do
7     With probability  $\epsilon$ , select a random action  $a_t$ ;
8     Otherwise select  $a_t = \max_a Q(\phi(s_t), a)$ ;
9     Execute action  $a_t$  in emulator and observe
       reward  $r_t$  and next states vectors  $x_{t+1}$  (here
        $x_{t+1} = x_t$ );
10    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess
        $\phi_{t+1} = \phi(s_{t+1})$ ;
11    Store transition  $(\phi_t, a_t, r_t, \phi'_{t+1})$  in  $\mathcal{D}$ ;
12    Sample random minibatch of transitions
        $(\phi_j, a_j, r_j, \phi'_{j+1})$  from  $\mathcal{D}$ ;
13    Set target  $y_j = r + \gamma \max_{a'} Q(\phi_{j+1}, a')$ ;
14    if  $MSELoss(y, Q(\phi, a)) < x$  AND
        $episode > \text{reset point}$  then
15      Perform a gradient descent step on
        $y_j - Q(\phi_j, a_j)$  according to  $MSELoss$ ;
16    end
17  end
18  if  $episode = \text{reset point}$  then
19    Reset  $\epsilon$  to 1 and gradually decay to 0.01 for a
       second time;
20  end
21  if  $episode = \text{flush point}$  then
22    Empty replay memory  $\mathcal{D}$ ;
23  end
24  Copy parameters of  $Q$  to  $Q_{\text{target}}$  every certain steps;
25 end

```

5) *One State DQN*: In this study, the application scenario does not involve state transitions; that is, after each decision made by the reinforcement learning agent, the state vector remains unchanged. Under this condition, the computation of the target Q-value is defined as follows:

$$\begin{aligned} \text{where } Q_{\text{target}} &= r + \gamma \cdot f_{\text{target}}(\text{states}) \\ f_{\text{target}}(\text{states}) &= \max_{a'} Q_{\text{target}}(\text{states}, a') \end{aligned} \quad (12)$$

where Q_{target} represents the Q-value of the target net, r denotes the immediate reward, γ is the discount factor, and $f_{\text{target}}(\text{states})$ denotes the Q-value obtained by inputting the current state into the target network, and f represents the mapping performed by the target network on the state.

This section proposes the OSDQN-RSF (One-State DQN with epsilon Reset, Selective Update, and Flushing Replay Buffer) method. The proposed OSDQN-RSF algorithm introduces four key improvements over the original DQN algorithm to customize the original algorithm for our satellite computational task scheduling problem. This section also introduces the definition of state space, action space, reward function and network architecture of the algorithm employed in this study.

The detailed pseudocode of the OSDQN-RSF algorithm is presented in Algorithm 1.

V. EVALUATION

All neural networks in this experiment were trained on a ground-based server. The server is equipped with two 20-core Intel Xeon Gold 6133 CPUs running at 2.50 GHz, four NVIDIA GeForce RTX 4090 GPUs, and operates on Ubuntu 23.04. The implementation is developed in Python 3.8.20 using the deep learning framework PyTorch 2.2.0+cu121, managed under Anaconda 24.9.2.

A. Parameter Settings

TABLE III
PARAMETER SETTINGS OF DIMENSIONLESS COEFFICIENTS ρ AND WEIGHTS ω IN THE COST QUANTIFICATION MODEL

Reward function parameters		
	ρ (dimensionless)	ω (weight)
On-orbit task		
Q (heat dissipation)	2	0.2
$FLOPs$ (computational load)	0.002	0.2
t_1 (task processing time)	0.002	0.1
t_2 (result transmission time)	0.002	0.1
$f(n)$ (device cost)	0.02	0.2
B (bandwidth occupancy)	2	0.2
Ground-based task		
Q (heat dissipation)	2	1/3
t (result transmission time)	0.0015	1/3
B (bandwidth occupancy)	2	1/3
Preprocessed on orbit $\rightarrow$ computed on the ground task		
Q_{pre} (data preprocessing heat dissipation)	2	0.2
Q_{trans} (data transmission heat dissipation)	2	0.2
t_{pre} (data preprocessing time)	0.002	0.2
t_{trans} (data transmission time)	0.002	0.2
$FLOPs$ (computational load)	0.002	0.2
B (bandwidth occupancy)	2	0.2

1) *Dimensionless Coefficients and Weight Assignments in Cost Quantification Modeling:* Since the cost quantification model requires the predefined configuration of dimensionless coefficients and weights, the parameter settings adopted in this

study are provided here for reference. The detailed configurations are presented in Table III.

The above table presents the settings of dimensionless coefficients and weights used in the cost quantification model during the experiments in this study. For different scheduling strategies with specific emphases, the weights of certain cost components can be adjusted accordingly, so that they are given more consideration during scheduling. Table III shows the parameter settings used in the experiments of this study and is provided for reference only.

2) *Co-location Cost Function Setting:* In the previous sections, the co-location cost was denoted by $f(n)$. In the experiments, the actual function used is defined as:

$$f(n) = n^2 \quad (13)$$

That is, the co-location cost increases quadratically with the number of tasks n colocated on the same satellite board.

B. Generation of Training and Testing Datasets

TABLE IV
RANDOM SAMPLING RANGES OF TRAINING NETWORK STATES AND PARAMETER VALUES

Parameter	Description	Random Sampling Range (unit: parameter's own unit)
Task on orbit		
Q	Heat generation	(0,1000)
$FLOPs$	Computation	(0,1000)
t_1	Task processing time	(0,1000)
t_2	Result transmission time	(200,300)
n	Number of cores	(0,50)
B	Bandwidth usage	(0,1000)
Task on ground		
Q	Heat generation	(0,1000)
t	Result transmission time	(200,20000)
B	Bandwidth usage	(0,1000)
Preprocessed on-orbit $\rightarrow$ ground model task		
Q_{pre}	Data pre-processing heat	(0,1000)
Q_{trans}	Data transmission heat	(0,1000)
t_{pre}	Data pre-processing time	(0,1000)
t_{trans}	Data transmission time	(0,1000)
$FLOPs$	Computation	(200,20000)
B	Bandwidth usage	(0,1000)

1) *Training Dataset Generation:* Since this study employs a reinforcement learning algorithm, there is no need for pre-labeled data. Instead, training data can be generated in batches over the state space. Therefore, during the network training phase, the datasets are produced by randomly sampling task parameters within reasonable ranges.

The specific generation method is summarized in Table IV.

2) *Testing Dataset Generation:* The generation of the testing dataset is similar to that of the training data, except that stricter bounds are applied to the variable ranges during generation. This ensures that the generated data values are more realistic and provide broad coverage across the state space for different scheduling strategies. Consequently, the testing dataset can accurately reflect the true performance of the trained network.

In this study, a total of 30,000 test instances are generated, with their distribution among the three scheduling strategies

(in the order described in Section III) approximately following a 4:3:3 ratio. The testing dataset provides good coverage of the state space values. The detailed test data can be found in the project source files.

C. Analysis of Experimental Results

TABLE V
HYPERPARAMETER SETTINGS FOR TRAINING THE OSDQN-RSF ALGORITHM

Hyperparameter	Value
Initial ϵ	1
ϵ_{decay}	0.9999
Learning rate	1×10^{-4}
Discount factor γ	0.9
Episodes	350,000 – 600,000
Replay buffer size	1,000,000 (deque)
Batch size	256
Loss maximum	15
Reset point	100,000
Flush point	200,000

In this section, the network architecture trained by the OSDQN-RSF algorithm is the same as illustrated in Section IV-B. After extensive hyperparameter tuning, the configuration that achieved the best performance was selected. The final training parameters are as follows: the initial value of ϵ was set to 1 with an ϵ_{decay} of 0.9999; the learning rate was 1×10^{-4} ; the discount factor γ was 0.9; the number of episodes was intermittently sampled between 350,000 and 600,000; the replay buffer size was implemented as a deque with a maximum length of 1,000,000; the batch size was 256; the loss maximum was constrained to 15; and the reset point and flush point were set to 100,000 and 200,000, respectively.

The model accuracy reported in Fig 8 was obtained using the self-constructed dataset introduced in Section V-B. As illustrated in Fig 8, under the above hyperparameter settings, the accuracy of the DQN-RSF algorithm remains relatively stable when the number of episodes ranges between 350,000 and 500,000, indicating that DQN-RSF is not highly sensitive to the number of training episodes and tends to converge easily. The best training run achieved a test set accuracy of 95.3%.

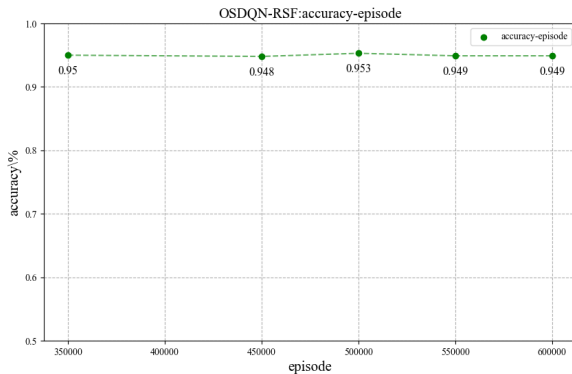


Fig. 8. Test set accuracy of the OSDQN-RSF algorithm under different numbers of training episodes.

D. Comparative Analysis of Cost Savings

TABLE VI
AVERAGE COSTS OF DIFFERENT ALGORITHMS ON TEST SET

	Q/J	t/ms	FLOPs/MFLOPs	B/Mbps
OSDQN-RSF	539.712	2679.510	419.392	338.674
On the Ground	570.540	11442.742	0	616.702
On Orbit	791.493	1041.580	791.480	572.935

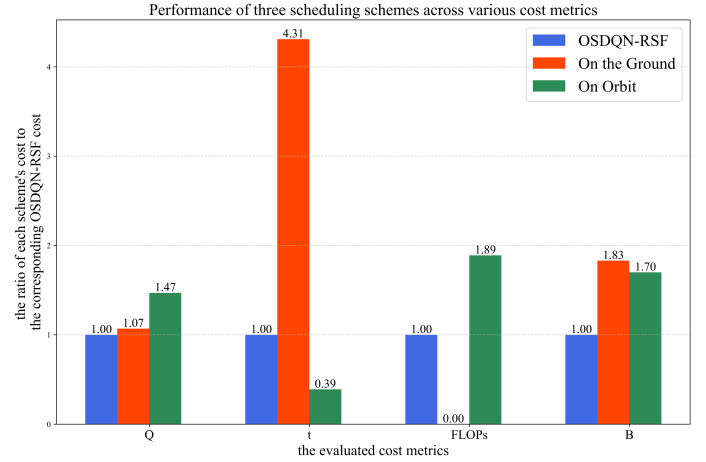


Fig. 9. Performance of different scheduling strategies across various cost metrics.

In the comparative analysis of cost savings, the OSDQN-RSF model with the highest accuracy was selected. The evaluation was conducted on 30,000 test samples, where the average values of multiple cost metrics were calculated, including onboard heat generation, processing latency, satellite-to-ground bandwidth occupancy, and onboard computational workload.

The results obtained from the OSDQN-RSF algorithm were compared against two baseline scenarios: no scheduling and processing tasks exclusively (i)in-orbit or (ii)on the ground. The comparative outcomes are summarized in Table VI and illustrated in Fig 9.

In Table VI, the terms *On the Ground* and *On Orbit* refer to non-reinforcement learning scheduling strategies, where all tasks are directly allocated to either the ground-based large model or the onboard small model for computation without any evaluation. In Figure 9, the FLOPs cost of assigning all tasks to the ground-based large model is zero because the satellite does not undertake any computation in this case, and all required data are transmitted to the ground for processing. Since the computational cost on the ground is not the focus of this study, it is reasonably ignored. However, this strategy of transmitting all raw task data to the ground without any preprocessing also results in a significantly higher time cost compared with other scheduling strategies.

By examining the figures, it is evident that, aside from the aforementioned special case, the application of reinforcement learning algorithms demonstrates substantial improvements in various cost metrics compared to strategies that directly assign tasks to the ground or satellite.

E. Summary

Based on the evaluation conducted on a test set containing 30,000 samples, the statistical analysis of the OSDQN-RSF algorithm demonstrates its effectiveness in reducing overall cost consumption. The proposed satellite task scheduling algorithm, built upon OSDQN-RSF, significantly decreases the utilization of satellite computing resources, optimizes task placement, and achieves a balance between task completion efficiency and operational cost.

Furthermore, the OSDQN-RSF algorithm enhances the training process of classic DQN algorithm, exhibiting superior performance in the satellite task scheduling scenario studied in this work. These findings also suggest its potential applicability to a broader range of reinforcement learning tasks beyond the present context.

VI. CONCLUSION

This study proposes a multi-dimensional cost evaluation model tailored to the characteristics of satellite computing, with a particular focus on key cost factors including thermal generation, latency, power consumption, and task co-location. The design of this model provides an effective tool for the quantitative analysis of the operational costs of different types of tasks in satellite computing environments, and lays a foundation for optimizing task allocation strategies as well as making rational decisions on whether to offload tasks to ground nodes or to satellite systems.

Building on the proposed quantitative cost modeling, this study further develops a scheduling algorithm between large-scale remote sensing models on the ground and small-scale remote sensing models onboard satellites, based on the improved **Deep Q-Network (DQN) algorithm**. Improvements upon the standard DQN framework lead to the proposed **OSDQN-RSF algorithm**. The OSDQN-RSF algorithm demonstrates excellent performance in the scheduling task, offering a feasible approach for task allocation between ground-based large models and satellite-based small models.

Experimental results indicate that the scheduling strategy based on the OSDQN-RSF algorithm reduces the consumption of satellite-side computational costs to one-half to one-quarter of those incurred when tasks are computed solely on the ground or solely on satellite, achieving an accuracy of **95.3%** on a self-constructed validation set. This enables more efficient scheduling of computational tasks between ground-based large models and satellite-based small models, thereby saving satellite computational resources and reducing various costs. In addition, the OSDQN-RSF algorithm shows potential for adaptation and application to other reinforcement learning problems.

REFERENCES

- [1] E. Casalicchio and D. Magliarisi, "Decentralized task scheduling in satellite edge computing," in *2024 9th International Conference on Fog and Mobile Edge Computing (FMEC)*, pp. 154–161, IEEE.
- [2] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," *arXiv preprint arXiv:1611.06440*, 2016.
- [3] J. Kim, J. Ahn, H.-L. Choi, and D.-H. Cho, "Task scheduling of agile satellites with transition time and stereoscopic imaging constraints," *Journal of Aerospace Information Systems*, vol. 17, no. 6, pp. 285–293, 2020.
- [4] X. Shen, Z. Lu, L. Li, Y. Chen, X. Li, J. Wang, and W. Yao, "Multi-strip stitching imaging mission planning method for sar satellites regional mapping considering on-board energy consumption," *IEEE Transactions on Geoscience and Remote Sensing*, 2024.
- [5] W. Yao, X. Shen, G. Zhang, Z. Lu, J. Wang, Y. Song, and Z. Li, "A spiking neural network based proximal policy optimization method for multi-point imaging mission scheduling of earth observation satellite," *Swarm and Evolutionary Computation*, vol. 94, p. 101867, 2025.
- [6] W. Liu, Z. Lai, Q. Wu, H. Li, Q. Zhang, Z. Li, Y. Li, and J. Liu, "In-orbit processing or not? sunlight-aware task scheduling for energy-efficient space edge computing networks," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*, pp. 881–890, IEEE.
- [7] Q. Jiang, H. Wang, Q. Kong, Y. Zhang, and B. Chen, "On-orbit remote sensing image processing complex task scheduling model based on heterogeneous multiprocessor," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 61, pp. 1–18, 2023.
- [8] Q. Jiang, L. Zheng, Y. Zhou, H. Liu, Q. Kong, Y. Zhang, and B. Chen, "Efficient on-orbit remote sensing imagery processing via satellite edge computing resource scheduling optimization," *IEEE Transactions on Geoscience and Remote Sensing*, 2025.
- [9] A. Abdulkarim, B. Maham, and R. C. Kizilirmak, "Multi-leo satellite networks for integrated access and backhaul: Delay analysis," in *2025 International Wireless Communications and Mobile Computing (IWCWC)*, pp. 1300–1305, IEEE, 2025.
- [10] A. Karkadakatil, "Ai-tuned hybrid thermal control of cubesats using phase change material: a matlab-based simulation study: Independent researcher, india," *Aerospace Systems*, pp. 1–10, 2025.
- [11] H. Zhang, H. Zhao, R. Liu, X. Gao, and S. Xu, "Dynamic user association and computation offloading in satellite edge computing networks via deep reinforcement learning," *IEEE Transactions on Green Communications and Networking*, vol. 8, no. 4, pp. 1888–1901, 2024.
- [12] X. Zhang, J. Liu, Z. Xiong, Y. Huang, R. Zhang, S. Mao, and Z. Han, "Cost-effective hybrid computation offloading in satellite-terrestrial integrated networks," *IEEE Internet of Things Journal*, vol. 11, no. 22, pp. 36786–36800, 2024.
- [13] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The journal of machine learning research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [14] R. Bellman, "Dynamic programming," *science*, vol. 153, no. 3731, pp. 34–37, 1966.
- [15] G. Tesauro, "Temporal difference learning and td-gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [16] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, pp. 279–292, 1992.